

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN MÔN HỌC
ĐỀ TÀI: HIỆN THỰC KHÓA SỐ ĐIỆN TỬ

Môn học: CE213.Q22 - Thiết kế hệ thống số với HDL

Giảng viên hướng dẫn: ThS. Hồ Ngọc Diễm

Thực hiện bởi nhóm bao gồm:

- | | | |
|-----------------------|----------|-------------|
| 1. Nguyễn Hữu Tình | 23521596 | Trưởng nhóm |
| 2. Lê Trần Thanh Toàn | 23521605 | Thành viên |
| 3. Nguyễn Minh Tiến | 23521578 | Thành viên |

Thời gian thực hiện: 04/2026 - 05/2026

MỤC LỤC

MỤC LỤC	2
DANH SÁCH HÌNH ẢNH	4
DANH SÁCH BẢNG.....	5
TÓM TẮT.....	7
Chương I. TỔNG QUAN	8
1. Giới thiệu đề tài	8
1.1. Tổng quan đề tài	8
1.2. Mục tiêu đề tài	8
2. Cơ sở lý thuyết.....	8
2.1. Ngôn ngữ mô tả phần cứng Verilog HDL	8
2.2. Hệ thống Kit phát triển Altera DE2 và công nghệ chip FPGA	9
2.3. Mô hình máy trạng thái hữu hạn FSM (Finite State Machine)	9
2.4. Công cụ thiết kế và mô phỏng (Quartus II & ModelSim).....	9
Chương II. THIẾT KẾ HỆ THỐNG	10
1. Phân tích phần cứng	10
1.1. Kit DE2 Altera	10
1.2. Matrix Keypad 4x4.....	11
1.3. Màn hình LCD.....	12
1.4. Buzzer.....	13
2. Kết nối các thành phần phần cứng.....	14
Chương III. TRIỂN KHAI HỆ THỐNG	16
1. Xây dựng sơ đồ khối.....	16
1.1. Khối Input.....	16
1.2. Khối xử lý trung tâm (khối xử lý trên FPGA).....	16
1.3. Khối Output	17
2. Xây dựng FSM	18
3. Xây dựng các module chính của hệ thống.....	19
3.1. Module lock_fsm	19
3.2. Module Keypad_Scanner	20

3.3. Module Password_manager.....	21
3.4. Module lcd_interface.....	21
3.5. Các module giao tiếp I2C (lcd_driver và i2c_write_byte).....	22
3.6. Module smart_lock.....	23
Chương IV. KẾT QUẢ THỰC NGHIỆM.....	26
1. Mô phỏng trên ModelSim	26
1.1. Mục đích mô phỏng.....	26
1.2. Nội dung testbench.....	26
1.3. Kịch bản kiểm tra	26
1.4. Mã testbench sử dụng.....	27
1.5. Kết quả mô phỏng	35
2. Kết quả nạp.....	36
CHƯƠNG V: KẾT LUẬN	41
1. Kết luận đề tài.....	41
2. Hướng phát triển.....	41

DANH SÁCH HÌNH ẢNH

Ảnh.1	Kit DE2 Altera	10
Ảnh.2	Matrix Keypad 4x4	11
Ảnh.3	Màn hình LCD 16x2	12
Ảnh.4	Buzzer	13
Ảnh.5	Sơ đồ chân của GPIO0 (GPIO1 tương tự).....	14
Ảnh.6	Sơ đồ khối của hệ thống	16
Ảnh.7	Mô hình FSM.....	18
Ảnh.8	7 trạng thái	19
Ảnh.9	Bộ định thời	19
Ảnh.10	Hàm giải mã.....	20
Ảnh.11	lưu mật khẩu mới nếu có tín hiệu	21
Ảnh.12	các thao tác với bộ đệm buffer.....	21
Ảnh.13	chuỗi hiển thị ra chỉ có dấu *	22
Ảnh.14	lcd driver	23
Ảnh.15	I2C write byte	23
Ảnh.16	Cấu hình Open-Drain cho I2C	24
Ảnh.17	Kết quả mô phỏng hiển thị console	36
Ảnh.18	Kết quả mô phỏng dạng sóng	36
Ảnh.19	Toàn bộ hệ thống Khóa số điện tử	37
Ảnh.20	Trạng thái đang nhập mã	37
Ảnh.21	Trạng thái đã mở cửa	38
Ảnh.22	Trạng thái chờ nhập mã mới (đổi mật khẩu)	38
Ảnh.23	Trạng thái đang nhập mã mới	39
Ảnh.24	Trạng thái nhập sai mật khẩu (lần 1 và 2).....	39
Ảnh.25	Trạng thái khóa vì nhập sai 3 lần.....	40

DANH SÁCH BẢNG

Bảng.1	Bảng phân công công việc	6
Bảng.2	Bảng kết nối ngoại vi và Kit DE2 Altera	14
Bảng.3	kịch bản kiểm tra	26

BẢNG PHÂN CÔNG NHIỆM VỤ

Bảng.1 Bảng phân công công việc

Họ và tên	Mã số sinh viên	Phân công công việc	Mức độ hoàn thành
Nguyễn Hữu Tình	23521596	Thiết kế module lock_fsm, password_manager, viết báo cáo, làm slide	100%
Lê Trần Thanh Toàn	23521605	Thiết kế module keypad_scanner, smart_lock, viết báo cáo, làm slide	100%
Nguyễn Minh Tiến	23521578	Thiết kế module lcd_driver, lcd_interface, viết báo cáo, làm slide	100%

TÓM TẮT

Đồ án tập trung vào thiết kế và hiện thực hệ thống Khóa số điện tử đơn giản sử dụng ngôn ngữ mô tả phân cứng Verilog HDL. Mục tiêu của đề tài là xây dựng một module điều khiển kiểm soát ra vào an toàn thông qua việc xác thực mật khẩu người dùng. Hệ thống được tổ chức thiết kế dựa trên mô hình máy trạng thái (FSM) để quản lý các chế độ hoạt động: chờ nhập, kiểm tra mật khẩu, mở khóa an toàn, đổi mật khẩu và cảnh báo khi nhập sai quá số lần quy định.

Toàn bộ mạch logic được kiểm tra tính đúng đắn thông qua hiện thực hóa phân cứng trên Kit DE2 Altera, kết hợp hệ thống LED, Buzzer và màn hình hiển thị để thông báo trạng thái trực quan. Kết quả đạt được là một hệ thống khóa số hoạt động ổn định theo thời gian thực, xử lý chính xác và linh hoạt các kịch bản bảo mật cơ bản, đáp ứng đầy đủ các yêu cầu kỹ thuật đề ra của môn học.

Chương I. TỔNG QUAN

1. Giới thiệu đề tài

1.1. Tổng quan đề tài

Trong xu thế phát triển của các hệ thống an ninh hiện đại, các phương thức khóa cơ học truyền thống đang dần được thay thế bằng các hệ thống khóa điện tử thông minh nhờ vào tính tiện dụng và độ an toàn cao. Đồ án môn học này tập trung vào việc nghiên cứu và hiện thực hóa một Hệ thống khóa số điện tử đơn giản sử dụng ngôn ngữ mô tả phần cứng Verilog HDL, đóng vai trò như một bộ kiểm soát ra vào an toàn. Hệ thống tiếp nhận mã số từ người dùng thông qua bàn phím matrix Keypad 4x4, thực hiện xử lý logic trung tâm, hiển thị trạng thái tương tác lên màn hình LCD, đồng thời điều khiển chốt khóa và kích hoạt còi báo Buzzer nếu phát hiện hành vi xâm nhập trái phép.

1.2. Mục tiêu đề tài

Đề tài đề ra ba mục tiêu chính như sau:

- Thiết kế cấu trúc module Verilog chạy đúng chuẩn logic yêu cầu, thực hiện phân chia module hợp lý.
- Mô phỏng thành công dạng sóng trên phần mềm để kiểm tra tính đúng đắn trước khi nạp xuống kit
- Hiện thực hóa phần cứng thành công trên Kit phát triển Altera DE2, gán chân chính xác và vận hành ổn định với các linh kiện ngoại vi kết nối qua cổng mở rộng GPIO.

2. Cơ sở lý thuyết

2.1. Ngôn ngữ mô tả phần cứng Verilog HDL

Verilog HDL (Hardware Description Language) là ngôn ngữ chuẩn công nghiệp được sử dụng để mô hình hóa, thiết kế và mô phỏng các hệ thống kỹ thuật số. Khác với các ngôn ngữ lập trình phần mềm thông thường vốn thực thi lệnh tuần tự (line-by-line), Verilog mô tả cấu trúc và hành vi của phần cứng, cho phép các khối mạch hoạt động đồng thời (concurrently).

Trong đồ án này, Verilog được sử dụng để viết mã nguồn cho toàn bộ hệ thống từ mức cấu trúc đến mức hành vi. Ngôn ngữ này giúp nhóm dễ dàng phân tách khóa số thành các khối driver độc lập (Driver LCD, Driver Keypad, Khối xử lý trung tâm), tạo điều kiện thuận lợi cho việc kiểm tra, sửa lỗi mạch logic.

2.2. Hệ thống Kit phát triển Altera DE2 và công nghệ chip FPGA

Kit phát triển Altera DE2 (sử dụng chip FPGA Cyclone II EP2C35F672C6) là một nền tảng mạnh mẽ dành cho giáo dục và thiết kế vi mạch. FPGA (Field-Programmable Gate Array) là loại vi mạch tích hợp có chứa hàng ngàn khối logic cấu hình được (LEs) và các kết nối nội vi linh hoạt, cho phép người kỹ sư tái cấu trúc lại sơ đồ mạch phần cứng ngay trên chip thông qua phần mềm lập trình.

Trong đồ án, Kit DE2 đóng vai trò là bo mạch chủ cung cấp tần số xung clock thạch anh ổn định (50 MHz), cung cấp các cổng mở rộng GPIO để kết nối với các module ngoại vi, và là nơi thực thi toàn bộ thư viện logic của hệ thống khóa số sau khi đã biên dịch.

2.3. Mô hình máy trạng thái hữu hạn FSM (Finite State Machine)

FSM là một mô hình toán học dùng để thiết kế các mạch tuần tự điều khiển có tư duy logic phức tạp. Tại một thời điểm, mạch chỉ có thể nằm ở một trạng thái duy nhất và sẽ dịch chuyển sang trạng thái tiếp theo dựa trên sự kết hợp của trạng thái hiện tại và tín hiệu kích hoạt đầu vào.

2.4. Công cụ thiết kế và mô phỏng (Quartus II & ModelSim)

Altera Quartus II là phần mềm chuyên dụng của Intel/Altera dùng để phân tích và cú pháp code Verilog, tối ưu hóa sơ đồ mạch logic, gán các chân vật lý của chip FPGA Cyclone II ra các chân cắm GPIO trên Kit DE2, và tiến hành nạp file cấu hình xuống board

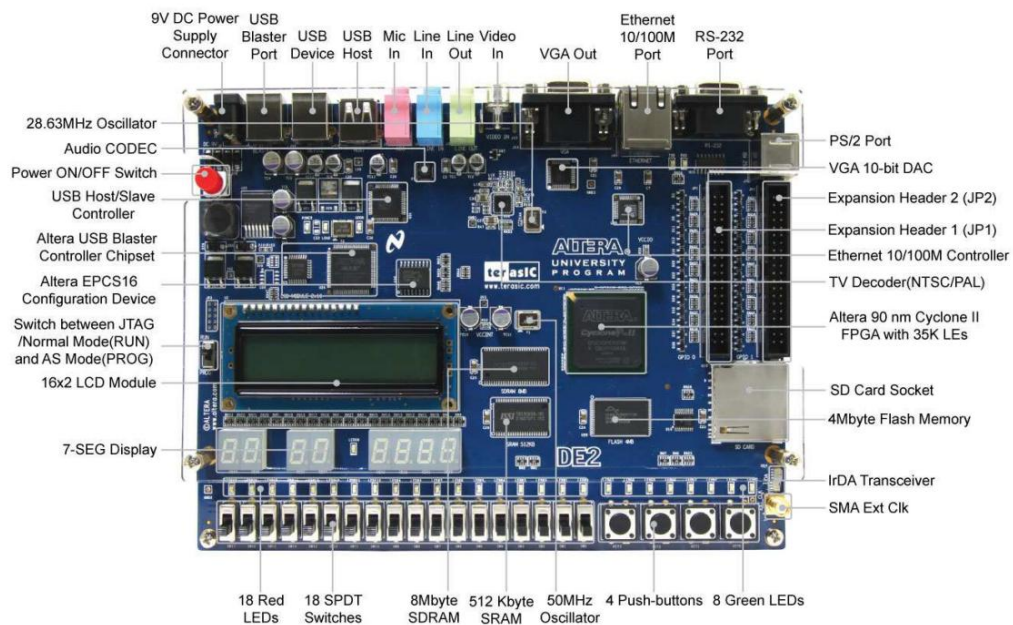
Modelsim là công cụ mô phỏng dạng sóng. Trước khi kết nối các linh kiện thật, nhóm sử dụng ModelSim để chạy Testbench, giả lập các tín hiệu nhấn phím Keypad nhằm quan sát xem FSM có chuyển trạng thái chính xác và các đường bus dữ liệu xuất ra LCD có đúng kịch bản hay không.

Chương II. THIẾT KẾ HỆ THỐNG

1. Phân tích phần cứng

1.1. Kit DE2 Altera

Bộ công cụ phát triển Altera DE2 là một nền tảng phần cứng mạnh mẽ và toàn diện, được thiết kế chuyên dụng cho việc học tập, nghiên cứu hệ thống số và công nghệ vi mạch định cấu hình. Trọng tâm của bo mạch là chip FPGA Cyclone II EP2C35F672C6 với hơn 33.000 phần tử logic (LEs), bộ nhớ nhúng dung lượng lớn và các khối quản lý xung clock (PLL) độc lập. Nhờ vào chip nạp USB Blaster tích hợp sẵn, người thiết kế có thể dễ dàng biên dịch mã nguồn từ phần mềm Quartus II và nạp trực tiếp file cấu hình xuống chip vật lý, biến các ý tưởng thiết kế trên ngôn ngữ Verilog thành mạch logic thực tế chạy với tốc độ phần cứng tối ưu theo thời gian thực.



Ảnh.1 Kit DE2 Altera

Bên cạnh chip xử lý trung tâm mạnh mẽ, Kit DE2 còn được trang bị hệ thống tài nguyên ngoại vi vô cùng phong phú và các cổng kết nối chuẩn công nghiệp. Bo mạch tích hợp sẵn các công tắc gạt, nút nhấn điều khiển cùng hệ thống LED đơn, LED 7 đoạn trực quan, đi kèm các bộ nhớ mở rộng (SDRAM, SRAM, Flash) và các cổng giao tiếp đa phương tiện như VGA, Audio, RS-232. Đặc biệt, đối với các đề tài cần mở rộng phần cứng, hai cổng kết nối GPIO 40-pin trên board đóng vai trò là cầu nối tín hiệu lý tưởng, cho phép chip FPGA dễ dàng giao tiếp, quét ma trận phím trên Keypad 4x4, xuất tập lệnh hiển thị ra LCD và điều khiển còi báo Buzzer một cách linh hoạt.

1.2. Matrix Keypad 4x4

Bàn phím ma trận Keypad 4x4 là một thiết bị ngoại vi nhập liệu phổ biến trong các hệ thống nhúng và mạch số, được cấu tạo từ 16 nút nhấn cơ học sắp xếp theo cấu trúc hình lưới gồm 4 hàng và 4 cột. Thay vì sử dụng 16 chân I/O độc lập gây lãng phí tài nguyên của chip điều khiển, sơ đồ mạch ma trận này khéo léo tối ưu hóa số lượng kết nối xuống chỉ còn 8 chân tín hiệu (4 chân hàng và 4 chân cột). Thiết kế này đòi hỏi hệ thống logic phải áp dụng kỹ thuật "Quét ma trận phím" (Matrix Scanning) bằng cách liên tục kéo thấp mức điện áp của từng hàng theo chu kỳ xung clock, đồng thời đọc giá trị phản hồi ở các cột để xác định chính xác tọa độ giao điểm của phím đang được nhấn.



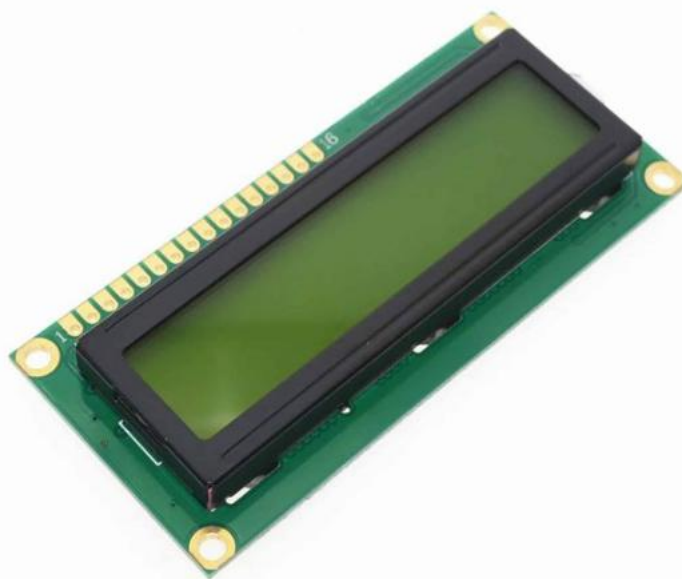
Ảnh.2 Matrix Keypad 4x4

Trong hệ thống khóa số điện tử, Keypad 4x4 đóng vai trò là giao diện tương tác trực tiếp giúp người dùng nhập các ký tự mật khẩu và câu lệnh điều khiển một cách dễ dàng. Để đảm bảo dữ liệu nhập vào chip FPGA trên Kit DE2 đạt độ chính xác tuyệt đối, mã nguồn Verilog không chỉ thực hiện giải mã tọa độ phím thành các ký tự số (từ 0 đến 9) hoặc ký tự chức năng (như *, #), mà còn phải tích hợp thêm khối mạch tuần tự điều khiển để chống rung phím (Debounce logic). Khối logic này giúp loại bỏ hoàn toàn các xung nhiễu cơ học phát sinh khi nhấn nút, ngăn chặn hiện tượng hệ thống

nhận diện sai hoặc nhận trùng lặp một ký tự nhiều lần, từ đó đảm bảo quá trình xác thực mật khẩu diễn ra mượt mà và an toàn.

1.3. Màn hình LCD 16x2 giao tiếp I2C

Màn hình LCD 16x2 là module hiển thị ký tự chữ và số phổ biến trong các hệ thống điện tử, có khả năng hiển thị 2 dòng, mỗi dòng tối đa 16 ký tự. Phần lớn các LCD 16x2 sử dụng bộ điều khiển tương thích chuẩn HD44780, hỗ trợ hiển thị ký tự thông qua bộ nhớ DDRAM và tập lệnh điều khiển cơ bản như xóa màn hình, bật/tắt hiển thị, đặt vị trí con trỏ và ghi dữ liệu ký tự.



Ảnh.3 Màn hình LCD 16x2

Trong đồ án này, LCD 16x2 không được điều khiển trực tiếp bằng giao tiếp song song RS, RW, E và bus dữ liệu D4–D7 từ FPGA. Thay vào đó, nhóm sử dụng module chuyển đổi I2C tích hợp chip mở rộng PCF8574 gắn phía sau LCD. Nhờ đó, FPGA chỉ cần sử dụng hai đường tín hiệu SDA và SCL để truyền dữ liệu, giúp giảm đáng kể số lượng chân GPIO cần dùng và làm hệ thống kết nối gọn hơn.

Về nguyên lý hoạt động, module `lcd_interface` sẽ nhận trạng thái hiện tại của hệ thống và số lượng ký tự mật khẩu đã nhập, sau đó tạo ra chuỗi ASCII 16 ký tự tương ứng để hiển thị lên LCD. Chuỗi này không hiển thị trực tiếp mật khẩu thật mà chỉ hiển thị các dấu “*” nhằm đảm bảo tính bảo mật khi người dùng nhập mã.

Chuỗi ký tự từ `lcd_interface` được đưa sang module `lcd_driver`. Module `lcd_driver` chịu trách nhiệm khởi tạo LCD ở chế độ 4-bit thông qua PCF8574, đặt con trỏ về đầu dòng hiển thị và lần lượt gửi 16 ký tự ASCII ra màn hình. Mỗi byte dữ liệu hoặc lệnh

điều khiển được tách thành hai nửa 4-bit, sau đó đóng gói cùng các bit điều khiển RS, RW, E và Backlight thành dữ liệu 8-bit gửi qua giao thức I2C. Như vậy, các tín hiệu RS, RW, E vẫn được sử dụng ở phía LCD, nhưng chúng không được FPGA điều khiển trực tiếp bằng các chân riêng lẻ mà được tạo gián tiếp thông qua chip PCF8574.

Trong thiết kế phần cứng, hai đường SDA và SCL được cấu hình theo kiểu open-drain bằng cách cho chân GPIO ở trạng thái kéo xuống mức 0 hoặc thả nổi mức Z. Cách cấu hình này phù hợp với đặc điểm của bus I2C và giúp LCD nhận dữ liệu ổn định trong quá trình hệ thống hoạt động.

1.4. Buzzer

Buzzer Piezo chủ động (Active Buzzer) hoạt động trong dải điện áp rộng từ 3V đến 24V, cho khả năng phát ra âm thanh cảnh báo liên tục với cường độ âm cực cao lên đến 95dB. Điểm vượt trội của loại còi báo Piezo này là tính bền bỉ, dòng tiêu thụ thấp và cơ chế vận hành vô cùng đơn giản. Do là Buzzer chủ động nên bên trong cấu kiện đã được tích hợp sẵn mạch tạo dao động; chỉ cần được cấp một mức điện áp DC phù hợp vào hai cực âm dương là còi sẽ tự động phát ra tiếng kêu cơ bản mà không đòi hỏi bộ điều khiển phải liên tục phát xung PWM phức tạp ở bên ngoài.



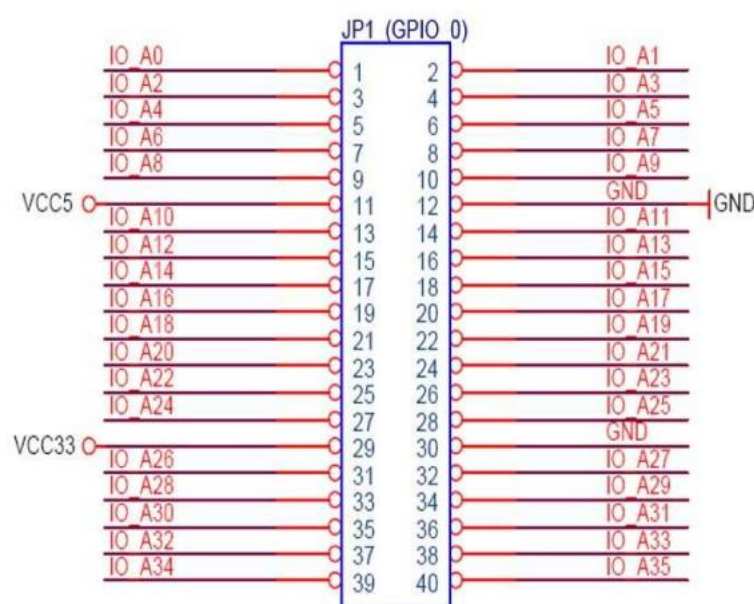
Ảnh.4 Buzzer

Trong kiến trúc của hệ thống khóa số điện tử, Buzzer đóng vai trò là thiết bị phản hồi âm thanh và phát tín hiệu cảnh báo khẩn cấp khi có sự cố xâm nhập trái

phép. Mặc dù có thể kêu trực tiếp bằng nguồn DC, mã nguồn Verilog trên Kit DE2 vẫn điều khiển linh hoạt bằng cách xuất tín hiệu mức logic ra chân GPIO để đóng/ngắt một transistor công suất (do dòng từ chân FPGA không đủ lớn để kích trực tiếp cho còi). Khối logic này sẽ phối hợp với máy trạng thái FSM để tạo ra kịch bản âm thanh rõ rệt kích hoạt còi kêu liên tục, vang dội khi hệ thống rơi vào trạng thái báo động do nhập sai mật khẩu quá số lần quy định để răn đe kẻ gian.

2. Kết nối các thành phần phần cứng

Để kết nối các thành phần phần cứng ngoại vi với Kit DE2, ta sẽ sử dụng các cổng GPIO0 và GPIO1 để thực hiện việc kết nối các thành phần này. Dưới đây là cấu tạo các chân của các GPIO (GPIO0 và GPIO1 đều có cấu tạo các chân giống nhau)



Ảnh.5 Sơ đồ chân của GPIO0 (GPIO1 tương tự)

Bảng dưới đây mô tả các thành phần phần cứng ngoại vi và cách kết nối với các chân GPIO (lưu ý khi kết nối Buzzer với GPIO0 phải kết nối với 1 điện trở):

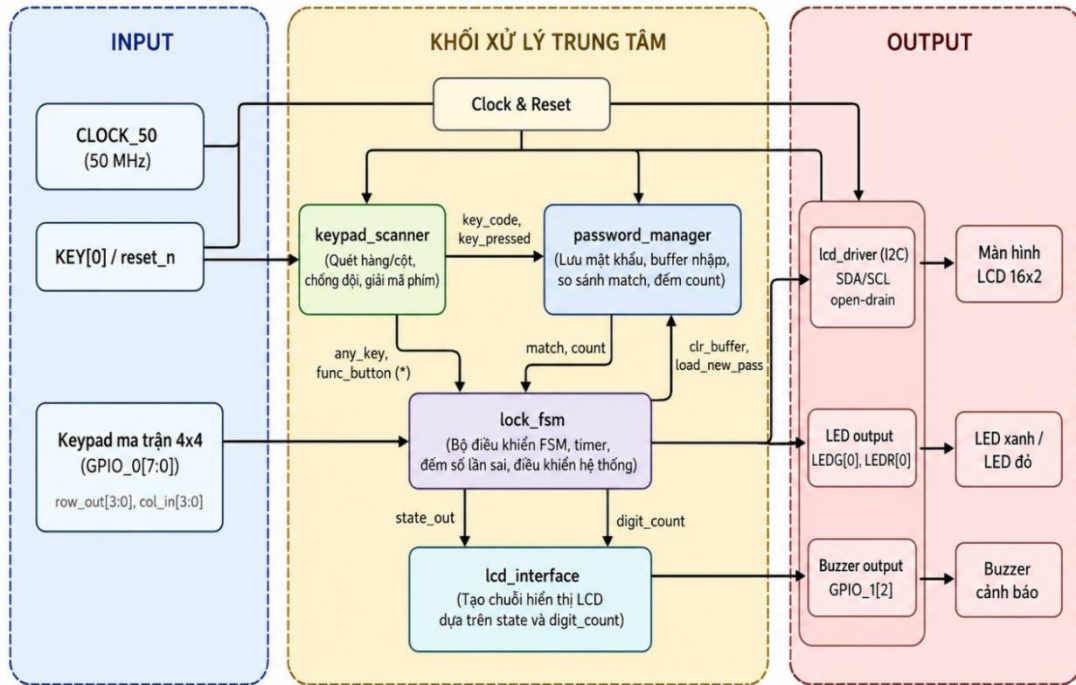
Bảng.2 Bảng kết nối ngoại vi và Kit DE2 Altera

Ngoại vi	Kết nối	Chân cụ thể
Matrix Keypad 4x4	GPIO0	R1 → GPIO_0[0]
		R2 → GPIO_0[1]

		R3 → GPIO_0[2]
		R4 → GPIO_0[3]
		C1 → GPIO_0[4]
		C2 → GPIO_0[5]
		C3 → GPIO_0[6]
		C4 → GPIO_0[7]
LCD 16x2	GPIO1	GND → GND
		VCC → VCC 5.5V
		SDA → GPIO_1[0]
		SCL → GPIO_1[1]
Buzzer	GPIO1	Dây đỏ/VDD → GPIO_1[2] qua điện trở 1kΩ
		Dây đen/GND → GND

Chương III. TRIỂN KHAI HỆ THỐNG

1. Xây dựng sơ đồ khối



Ảnh.6 Sơ đồ khối của hệ thống

Sơ đồ khối của hệ thống được xây dựng bao gồm 3 khối chức năng chính:

1.1. Khối Input

Khối INPUT là nơi tiếp nhận thông tin từ người dùng và môi trường phần cứng để gửi vào hệ thống xử lý. Khối này bao gồm ba thành phần chính: xung CLOCK_50 50MHz cấp nguồn nhịp cho toàn mạch, nút nhấn KEY[0] / reset_n đảm nhận nhiệm vụ thiết lập lại trạng thái ban đầu, và bàn phím ma trận Keypad 4x4 kết nối qua bus GPIO_0[7:0] (gồm 4 đường quét hàng row_out[3:0] và 4 đường đọc cột col_in[3:0]). Các tín hiệu đầu vào này được định tuyến trực tiếp vào các module chức năng tương ứng trong chip FPGA mà không cần qua module đệm trung gian.

1.2. Khối xử lý trung tâm (khối xử lý trên FPGA)

Khối xử lý trung tâm đóng vai trò đảm nhận vai trò quản lý logic, kiểm tra dữ liệu và điều phối toàn bộ hoạt động của khóa số thông qua module nội vi:

- Clock và reset: tiếp nhận xung clock gốc 50 MHz và tín hiệu reset_n để thực hiện chia tần số, cung cấp xung nhịp đồng bộ ổn định và quản lý trạng thái khởi tạo an toàn cho toàn bộ hệ thống

- Keypad_scanner: nhận tín hiệu từ keypad ma trận 4x4, thực hiện chức năng quét hàng - cột, chống dội và giải mã. Module này xuất ra phím (key_code), xung báo nhấn phím (key_pressed) cho khối quản lý mật mã, đồng thời gửi tín hiệu phát hiện phím bất kỳ (any_key) và phím chức năng đổi mật khẩu (func_button (*)) trực tiếp cho bộ điều khiển FSM
- Password_manager: đóng vai trò là đường truyền dữ liệu, chịu trách nhiệm lưu trữ mật mã gốc, quản lý bộ đếm nhập liệu từ bàn phím, đếm số ký tự đã nhập thành công và thực hiện so sánh logic để đưa ra kết quả trung khớp
- Lock_fsm: là khối điều khiển trung tâm sử dụng máy trạng thái kết hợp bộ định thời và bộ đếm số lần nhập sai. Khối này nhận tín hiệu điều khiển để quyết định các bước chuyển trạng thái.
- Lcd_interface: nhận dữ liệu trạng thái và số lượng phím đã nhập từ lock_fsm để biên dịch thành chuỗi văn bản ASCII 128-bit.

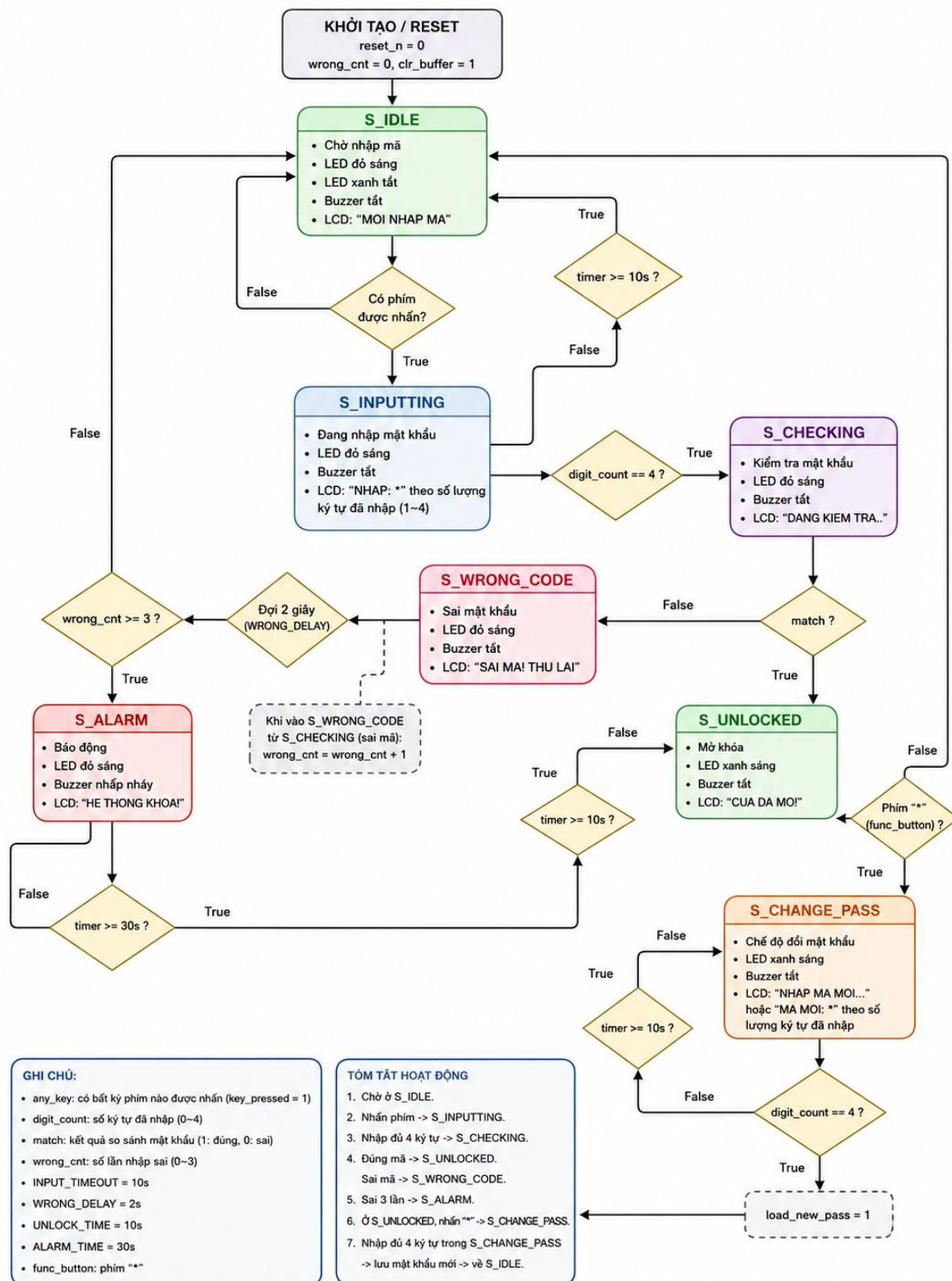
1.3. Khối Output

Khối OUTPUT có nhiệm vụ tiếp nhận kết quả xử lý từ FPGA để chuyển đổi thành các tín hiệu tương tác vật lý trực quan ra môi trường ngoài, bao gồm 2 phân tầng:

- Các module Driver trung gian: Gồm lcd_driver (I2C) cấu hình đường truyền SDA/SCL open-drain để biên dịch chuỗi ký tự từ lcd_interface sang chuẩn giao tiếp nối tiếp; mạch LED output nhận lệnh điều khiển hai chân LEDG[0] và LEDR[0]; cùng mạch Buzzer output định tuyến tín hiệu ra chân mở rộng GPIO_1[2].
- Các thiết bị ngoại vi vật lý: Nhận dòng kích hoạt từ mạch driver để phản hồi cho người dùng, bao gồm Màn hình LCD 16x2 để hiển thị thông báo bằng ngôn ngữ tự nhiên, hệ thống LED xanh / LED đỏ để hiển thị nhanh trạng thái chốt cửa (Đóng/Mở), và Buzzer cảnh báo phát tín hiệu còi vang dội khi phát hiện hành vi xâm nhập trái phép

2. Xây dựng FSM

Bên dưới là mô hình FSM được xây dựng trong đồ án lần này:



Ảnh.7 Mô hình FSM

3. Xây dựng các module chính của hệ thống

Hệ thống được xây dựng bao gồm 6 khối module chính:

3.1. Module lock_fsm

Module lock_fsm đóng vai trò bộ não điều khiển toàn bộ hệ thống. Module này chứa máy trạng thái hữu hạn để quản lý luồng hoạt động của khóa số, tính toán thời gian trễ, đếm số lần nhập sai và đưa ra các quyết định đóng/mở chốt cửa hoặc kích hoạt báo động.

Thành phần logic cốt lõi gồm:

- Quản lý 7 trạng thái hệ thống: S_IDLE (chờ), S_INPUTTING (nhập số), S_CHECKING (kiểm tra mã), S_UNLOCKED (mở khóa), S_WRONG_CODE (báo sai mã), S_CHANGE_PASS (đổi mật khẩu), và S_ALARM (báo động khẩn cấp).

```
localparam S_IDLE           = 3'd0;
localparam S_INPUTTING      = 3'd1;
localparam S_CHECKING       = 3'd2;
localparam S_UNLOCKED       = 3'd3;
localparam S_WRONG_CODE     = 3'd4;
localparam S_ALARM          = 3'd5;
localparam S_CHANGE_PASS    = 3'd6;
```

Ảnh.8 7 trạng thái

- Bộ định thời (Timer 32-bit): đếm theo xung clock 50MHz để tạo các khoảng trễ chính xác: 10 giây tự động hủy trạng thái nếu người dùng quên không bấm phím; 2 giây giữ màn hình thông báo sai mã; 30 giây khóa cứng màn hình khi có báo động

```
localparam INPUT_TIMEOUT    = 32'd500_000_000;
localparam UNLOCK_DELAY     = 32'd500_000_000;
localparam WRONG_DELAY      = 32'd100_000_000;
localparam ALARM_DELAY      = 32'd1_500_000_000;
```

Ảnh.9 Bộ định thời

- Bộ đếm sai mật khẩu: kiểm tra số lần sai liên tiếp đạt ngưỡng giới hạn (WRONG_LIMIT = 3), FSM lập tức cưỡng bức hệ thống chuyển sang trạng thái S_ALARM

- Điều khiển LED và Buzzer: điều khiển nếu hệ thống mở cửa thì LED xanh bật, nếu nhập khóa cửa thì LED đỏ bật, đồng thời Buzzer cũng sẽ báo động

3.2. Module Keypad_Scanner

Module Keypad_Scanner chịu trách nhiệm giao tiếp vật lý và giải mã ma trận phím bấm Keypad 4x4 cắm. Module chuyển đổi trạng thái đóng/ngắt cơ học tại các giao điểm Hàng-cột thành một mã phím nhị phân 4-bit ổn định và tạo xung báo hiệu cho hệ thống.

Thành phần logic cốt lõi gồm:

- Kỹ thuật quét tuần tự: Module liên tục kéo thấp lần lượt điện áp của từng hàng (row_out từ R1 tới R4) cách nhau mỗi (ROW_TICKS). Sau khi đổi hàng 100 us, (SETTLE_TICKS), tín hiệu cột (col_in) sẽ được lấy mẫu để đảm bảo đường truyền đã ổn định.
- Hàm giải mã (decode_key): ánh xạ trực tiếp vị trí hàng - cột đang nhấp thành mã HEX 4-bit tương ứng (từ 0 đến 9, A đến D, phím * thành 4'hE, phím # thành 4'F)

```
function [4:0] decode_key;
input [1:0] row;
input [3:0] col;
begin
    decode_key = {1'b0, 4'h0};

    case (row)
        // Hàng 1: 1 2 3 A
        2'd0: begin
            if (col[0]) decode_key = {1'b1, 4'h1};
            else if (col[1]) decode_key = {1'b1, 4'h2};
            else if (col[2]) decode_key = {1'b1, 4'h3};
            else if (col[3]) decode_key = {1'b1, 4'hA};
        end

        // Hàng 2: 4 5 6 B
        2'd1: begin
            if (col[0]) decode_key = {1'b1, 4'h4};
            else if (col[1]) decode_key = {1'b1, 4'h5};
            else if (col[2]) decode_key = {1'b1, 4'h6};
            else if (col[3]) decode_key = {1'b1, 4'hB};
        end
    end
```

Ảnh.10 Hàm giải mã

- Khôi lọc chống nhiễu (Debounce Logic): tiến hành lưu mẫu và đối chiếu dữ liệu liên tục trong 5 khung quét. Tín hiệu này chỉ được công nhận là phím nhấn hợp lệ khi và chỉ khi nó giữ nguyên trạng thái liên tục qua thời gian lọc, giúp loại bỏ hiện tượng rung cơ học của phím bấm. Xuất ra xung key_pressed đúng 1 chu kỳ clock cho mỗi lần bấm phím.

3.3. Module Password_manager

Module Password_manager đảm nhận vai trò lưu trữ dữ liệu mật mã gốc và quản lý bộ đệm nhập liệu của người. Module này hoạt động dưới sự điều phối hành động của của khối điều khiển FSM.

Thành phần logic cốt lõi gồm:

- Thanh ghi mật mã gốc (saved_pass): mật mã mặc định khi hệ thống reset là 16'h1234. Khi nhận được tín hiệu load_new_pass từ FSM, thanh ghi này sẽ chốt giá trị mới từ bộ đệm để thực hiện đổi mật khẩu thành công.

```
always @(posedge clk or negedge reset_n)
    if (!reset_n)
        saved_pass <= 16'h1234;
    else
        if (load_new_pass)
            saved_pass <= buffer;
```

Ảnh.11 lưu mật khẩu mới nếu có tín hiệu

- Thanh ghi bộ đệm (buffer 16-bit): mỗi khi có xung key_pressed báo có phím số hợp lệ, dữ liệu trong bộ đệm sẽ thực hiện phép dịch trái 4 bit để nhường vị trí thấp nhất cho ký tự vừa mới nhập, đồng thời tăng bộ đếm chữ số từ 0 lên tối đa 4.

```
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        buffer <= 16'h0; count <= 3'd0;
    end
    else
        if (clr_buffer) begin
            buffer <= 16'h0; count <= 3'd0;
        end
        else
            if (key_pressed && count < 3'd4) begin
                // Dịch chuyển bit để nhận phím mới
                buffer <= {buffer[11:0], key_in};
                count <= count + 3'd1;
            end
        end
    end
    always @(*) begin
        match = (buffer == saved_pass) && (count == 3'd4);
    end
endmodule
```

Ảnh.12 các thao tác với bộ đệm buffer

3.4. Module lcd_interface

Module lcd_interface đóng vai trò là mạch dịch ký tự tương thích với trạng thái hệ thống. Module này nhận thông tin trạng thái hiện tại (state) và số lượng phím đã

nhân (digit_count) để biên dịch thành một chuỗi văn bản ASCII 16 ký tự tương ứng gửi cho driver hiển thị.

Thành phần logic cốt lõi gồm:

- Sử dụng kiến trúc mạch tổ hợp thuần túy (always @(*)) với cấu trúc case-inside-case để tối ưu hóa tốc độ chuyển đổi dữ liệu.
- Tự động mã hóa ẩn mật khẩu khi người dùng đang thao tác: dựa trên biến digit_count, module sẽ thay đổi linh hoạt chuỗi hiển thị tương ứng từ 1 đến 4 dấu * tại các chế độ S_INPUTTING hoặc S_CHANGE_PASS. Các trạng thái tĩnh sẽ được gán chuỗi thông báo khác.

```
always @(*) begin
    case (state)

        3'd0: lcd_string = "MOI NHAP MA      "; // S_IDLE

        3'd1: begin // S_INPUTTING
            case (digit_count)
                3'd1: lcd_string = "NHAP: *          ";
                3'd2: lcd_string = "NHAP: **         ";
                3'd3: lcd_string = "NHAP: ***        ";
                3'd4: lcd_string = "NHAP: ****       ";
                default: lcd_string = "NHAP:           ";
            endcase
        end
    end
```

Ảnh.13 chuỗi hiển thị ra chỉ có dấu *

3.5. Các module giao tiếp I2C (lcd_driver và i2c_write_byte)

Đây là cụm module đảm nhiệm giao thức truyền thông nối tiếp I2C để FPGA giao tiếp dữ liệu với chip mở rộng được gắn tích hợp phía sau màn hình LCD 16x2.

Thành phần logic cốt lõi gồm:

- Lcd_driver: tổ chức một FSM phức tạp điều khiển LCD chạy ở chế độ 4-bit. Quá trình bao gồm việc gửi chuỗi cấu hình khởi tạo đặc biệt (3, 3, 3, 2 để LCD vào chế độ 4-bit), định vị con trỏ về đầu dòng, tách 1 byte dữ liệu ASCII thành hai nửa rồi đóng gói cùng các bit điều khiển thông qua hàm pcf_byte để ra lệnh cho driver I2C cấp thấp gửi đi.

```
module lcd_driver #(
    parameter [6:0] I2C_ADDR = 7'h27 // N
) (
    input clk,                // CLOCK_50
    input reset_n,            // KEY[0]
    input [127:0] data_in,    // Chuoi 16 k

    // I2C open-drain signals ra top-level
    input  sda_in,
    output sda_drive_low,
    output scl_drive_low
);
```

Ảnh.14 lcd driver

- I2C_write_byte: triển khai vật lý chính xác các phân đoạn của giao thức I2C tiêu chuẩn. Module thực thi bộ chia tần Divider để hạ băng thông xung truyền về mức an toàn, quản lý chính xác các mốc thời gian tạo điều kiện, dịch tuần tự 8 bit dữ liệu,...

```
module i2c_write_byte #(
    parameter integer DIVIDER = 250
) (
    input clk,
    input reset_n,

    input start,
    input [6:0] addr,
    input [7:0] data,

    input sda_in,

    output reg busy,
    output reg done,

    output reg sda_drive_low,
    output reg scl_drive_low
);
```

Ảnh.15 I2C write byte

3.6. Module smart_lock

Module smart_lock là module cấp cao nhất, đóng vai trò là sơ đồ mạch in tổng kết nối, liên kết tất cả các module có chức năng độc lập ở trên lại với nhau thông qua hệ thống dây nối nội bộ (wire). Đồng thời, đây là nơi trực tiếp định cấu hình gán chân I/O để ánh xạ các biến phần ra các linh kiện vật lý thực tế trên Kit Altera DE2.

Thành phần logic cốt lõi:

- Cấu hình Open-Drain cho I2C: sử dụng kiến trúc gán chân ba trạng thái trên đường bus GPIO_1 để tạo ra các cổng hở dòng Open-Drain đúng chuẩn giao tiếp I2C, bảo vệ an toàn điện áp cho các chân của chip FPGA Cyclone II.

```
wire sda_drive_low;  
wire scl_drive_low;  
  
assign GPIO_1[0] = sda_drive_low ? 1'b0 : 1'bz; // SDA open-drain  
assign GPIO_1[1] = scl_drive_low ? 1'b0 : 1'bz; // SCL open-drain
```

Ảnh.16 Cấu hình Open-Drain cho I2C

- Quản lý luồng ngoại vi: ánh xạ chân KEY[0] của board làm tín hiệu Reset hệ thống, gán tín hiệu điều khiển của FSM ra cổng còi báo GPIO_1[2], và gán trực tiếp 2 ngõ ra trạng thái khóa cửa ra hai hệ thống LED vật lý trên.

Chương IV. KẾT QUẢ THỰC NGHIỆM

1. Mô phỏng trên ModelSim

1.1. Mục đích mô phỏng

Mục đích của phần mô phỏng là kiểm tra nhanh các chức năng chính của code thiết kế hệ thống khóa cửa thông minh trước khi đưa vào báo cáo tổng. Testbench tạo xung clock, reset hệ thống và giả lập thao tác nhấn phím để kiểm tra các trạng thái hoạt động của mạch.

Các tín hiệu được quan sát chính gồm: CLOCK_50, KEY, LEDG, LEDR, GPIO_0, GPIO_1, state, wrong_cnt, timer, saved_pass, buffer, match, lcd_string.

1.2. Nội dung testbench

Testbench sử dụng top module tb_smart_lock để mô phỏng toàn bộ hệ thống. Để rút ngắn thời gian mô phỏng, testbench ép trực tiếp hai tín hiệu nội bộ w_key_code và w_key_pressed nhằm giả lập phím được nhấn, thay vì chờ quá trình quét và chống dội phím thật từ keypad.

Ngoài ra, testbench có các task bỏ qua thời gian chờ như WRONG_DELAY và UNLOCK_DELAY bằng cách tăng nhanh timer đến đúng ngưỡng cần thiết. Cách này giúp kiểm tra FSM nhanh hơn nhưng vẫn giữ đúng logic chuyển trạng thái.

1.3. Kịch bản kiểm tra

Bảng.3 kịch bản kiểm tra

Test	Thao tác mô phỏng	Kết quả mong đợi
TEST 1	Nhập mật khẩu mặc định 1234	Hệ thống mở cửa, LED xanh sáng, LCD hiển thị CUA DA MO!
TEST 2	Nhấn * và nhập mật khẩu mới 5678	Mật khẩu lưu được cập nhật thành 5678
TEST 3	Nhập lại mã cũ 1234	Hệ thống báo sai mã, không mở cửa
TEST 4	Nhập mã mới 5678	Hệ thống mở cửa với mật khẩu mới

TEST 5	Nhập sai 3 lần: 0000, 1111, 2222	FSM vào S_ALARM, buzzer được kích hoạt
--------	----------------------------------	--

1.4. Mã testbench sử dụng

Dưới đây là testbench dùng để chạy mô phỏng kiểm tra chức năng:

```
`timescale 1ns/1ps

module tb_smart_lock;

    reg CLOCK_50;
    reg [3:0] KEY;
    wire [35:0] GPIO_0;
    wire [35:0] GPIO_1;
    wire [8:0] LEDG;
    wire [17:0] LEDR;

    smart_lock dut (
        .CLOCK_50(CLOCK_50),
        .KEY(KEY),
        .GPIO_0(GPIO_0),
        .GPIO_1(GPIO_1),
        .LEDG(LEDG),
        .LEDR(LEDR)
    );
```

```
// Clock 50 MHz, chu kỳ 20 ns

initial CLOCK_50 = 1'b0;

always #10 CLOCK_50 = ~CLOCK_50;


// Các giá trị delay trong lock_fsm

// Dùng dung ngưỡng delay, không force 32'hFFFF_FFFF để tránh vượt qua
S_ALARM.

localparam [31:0] WRONG_DELAY_VALUE = 32'd100_000_000;

localparam [31:0] UNLOCK_DELAY_VALUE = 32'd500_000_000;


// Mã trạng thái trong lock_fsm

localparam [2:0] S_IDLE      = 3'd0;

localparam [2:0] S_INPUTTING = 3'd1;

localparam [2:0] S_CHECKING  = 3'd2;

localparam [2:0] S_UNLOCKED  = 3'd3;

localparam [2:0] S_WRONG_CODE = 3'd4;

localparam [2:0] S_ALARM      = 3'd5;

localparam [2:0] S_CHANGE_PASS = 3'd6;


// Tạo 1 xung phím nhấn bằng cách force tín hiệu nội bộ của top

task press_key;

    input [3:0] key;

    begin
```

```
@(negedge CLOCK_50);

force dut.w_key_code  = key;

force dut.w_key_pressed = 1'b1;

@(negedge CLOCK_50);

force dut.w_key_pressed = 1'b0;

repeat (3) @(negedge CLOCK_50);

end

endtask

task enter_code;

input [3:0] k1, k2, k3, k4;

begin

    press_key(k1);

    press_key(k2);

    press_key(k3);

    press_key(k4);

    repeat (5) @(negedge CLOCK_50);

end

endtask

// Tang nhanh timer de FSM thoat khoi trang thai delay.

// Luu y: force dung nguoi can thiet, sau do release ngay sau canh clock.

// Neu force 32'hFFFF_FFFF, khi vao S_ALARM timer van qua lon va FSM co the
thoat S_ALARM ngay.
```

```
task jump_timer;

    input [31:0] value;

    begin

        @(negedge CLOCK_50);

        force dut.controller_inst.timer = value;

        @(posedge CLOCK_50);

        #1;

        release dut.controller_inst.timer;

        repeat (3) @(negedge CLOCK_50);

    end

endtask


task skip_wrong_delay;

    begin

        jump_timer(WRONG_DELAY_VALUE);

    end

endtask


task skip_unlock_delay;

    begin

        jump_timer(UNLOCK_DELAY_VALUE);

    end

endtask
```

```
task show_status;

    input [160*8:1] msg;

    begin

        $display("[%0t ns] %0s | state=%0d wrong_cnt=%0d timer=%0d count=%0d
match=%0b LEDG0=%0b LEDR0=%0b buzzer=%0b saved_pass=%0h buffer=%0h
lcd='%s'",

            $time, msg,

            dut.controller_inst.state_out,

            dut.controller_inst.wrong_cnt,

            dut.controller_inst.timer,

            dut.pass_inst.count,

            dut.pass_inst.match,

            LEDG[0], LEDR[0], GPIO_1[2],

            dut.pass_inst.saved_pass,

            dut.pass_inst.buffer,

            dut.w_lcd_string);

    end

endtask

initial begin

    $display("==== BAT DAU MO PHONG SMART LOCK =====");

    // Reset he thong, KEY[0] active-low

    KEY = 4'b1110;

    repeat (5) @(negedge CLOCK_50);
```

```
KEY[0] = 1'b1;

repeat (5) @(negedge CLOCK_50);

show_status("Sau reset");


// TEST 1: Nhập mật khẩu mặc định 1234 -> mở khóa

$display("\n--- TEST 1: Nhập đúng mật khẩu 1234 ---");

enter_code(4'h1, 4'h2, 4'h3, 4'h4);

repeat (5) @(negedge CLOCK_50);

show_status("Sau khi nhập 1234");


if (dut.controller_inst.state_out == S_UNLOCKED && LEDG[0] == 1'b1)

    $display("PASS TEST 1: Đúng mật khẩu -> cửa mở, LED xanh sáng.");

else

    $display("FAIL TEST 1: Chưa vào S_UNLOCKED.");


// TEST 2: Khi đang mở khóa, bấm '*' rồi nhập mã mới 5678

$display("\n--- TEST 2: Đổi mật khẩu mới thành 5678 ---");

press_key(4'hE); // phím '*'

repeat (5) @(negedge CLOCK_50);

show_status("Sau khi bấm * để vào chế độ đổi mã");


enter_code(4'h5, 4'h6, 4'h7, 4'h8);

repeat (5) @(negedge CLOCK_50);

show_status("Sau khi nhập mã mới 5678");
```

```
if (dut.pass_inst.saved_pass == 16'h5678)

    $display("PASS TEST 2: Da luu mat khau moi 5678.");

else

    $display("FAIL TEST 2: Chua luu dung mat khau moi.");

// TEST 3: Nhap lai ma cu 1234 -> sai

$display("\n--- TEST 3: Nhap lai ma cu 1234, yeu cau phai sai ---");

enter_code(4'h1, 4'h2, 4'h3, 4'h4);

repeat (5) @(negedge CLOCK_50);

show_status("Sau khi nhap lai ma cu 1234");

if (dut.controller_inst.state_out == S_WRONG_CODE)

    $display("PASS TEST 3: Ma cu bi tu choi, vao S_WRONG_CODE.");

else

    $display("FAIL TEST 3: Ma cu khong bi tu choi nhu mong doi.");

skip_wrong_delay();

show_status("Sau khi bo qua WRONG_DELAY");

// TEST 4: Nhap ma moi 5678 -> mo khoa

$display("\n--- TEST 4: Nhap ma moi 5678 ---");

enter_code(4'h5, 4'h6, 4'h7, 4'h8);

repeat (5) @(negedge CLOCK_50);
```

```
show_status("Sau khi nhap ma moi 5678");

if (dut.controller_inst.state_out == S_UNLOCKED && LEDG[0] == 1'b1)
    $display("PASS TEST 4: Mat khau moi dung -> cua mo.");
else
    $display("FAIL TEST 4: Mat khau moi khong mo khoa.");

// TEST 5: Nhap sai 3 lan -> bao dong
$display("\n--- TEST 5: Sai 3 lan de kiem tra bao dong ---");
skip_unlock_delay(); // tu S_UNLOCKED ve S_IDLE
show_status("Sau khi bo qua UNLOCK_DELAY, san sang test bao dong");

enter_code(4'h0, 4'h0, 4'h0, 4'h0);
repeat (5) @(negedge CLOCK_50);
show_status("Sai lan 1");
skip_wrong_delay();
show_status("Sau WRONG_DELAY lan 1");

enter_code(4'h1, 4'h1, 4'h1, 4'h1);
repeat (5) @(negedge CLOCK_50);
show_status("Sai lan 2");
skip_wrong_delay();
show_status("Sau WRONG_DELAY lan 2");
```

```
enter_code(4'h2, 4'h2, 4'h2, 4'h2);

repeat (5) @(negedge CLOCK_50);

show_status("Sai lan 3");

skip_wrong_delay();

show_status("Sau WRONG_DELAY lan 3");


if (dut.controller_inst.state_out == S_ALARM)

    $display("PASS TEST 5: Sai 3 lan -> vao S_ALARM.");

else

    $display("FAIL TEST 5: Chua vao S_ALARM. Kiem tra wrong_cnt va timer
tren waveform.");


    $display("\n===== KET THUC MO PHONG =====");

    #100;

    $stop;

end

endmodule
```

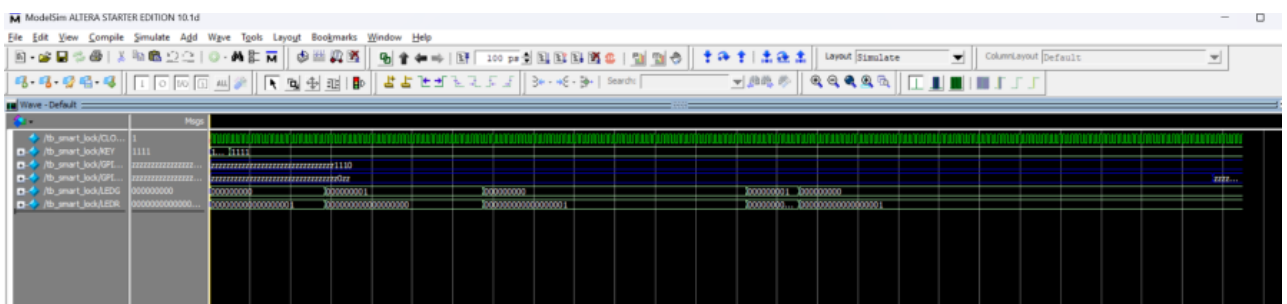
1.5. Kết quả mô phỏng

Sau khi chạy lệnh run -all trên ModelSim, cửa sổ transcript hiển thị các kết quả PASS cho toàn bộ các kịch bản kiểm tra. Điều này cho thấy các chức năng chính của hệ thống đã hoạt động đúng theo yêu cầu.

```
# .main_pane.objects.interior.ca.body.tree
# run -all
===== BẮT ĐẦU MÔ PHỎNG SMART LOCK =====
[2000000 ns] Sau reset : state=0 wrong_cnt=0 timer=5 count=0 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=1234 buffer=0000 lcd="MỜI NHẬP MÃ"
--- TEST 1: Nhập đúng mật khẩu 1234 ---
[8000000 ns] Sau khi nhập 1234 : state=3 wrong_cnt=0 timer=11 count=4 match=1 LEDG0=1 LEDR0=1 buzzer=0 saved_pass=1234 buffer=1234 lcd="CỬA ĐÃ MỞ!"
PASS TEST 1: Dung mật khẩu -> cửa mở, LED xanh sáng.
--- TEST 2: Đổi mật khẩu mới thành 5678 ---
[10000000 ns] Sau khi ban 'de vao che do doi ma' : state=4 wrong_cnt=0 timer=8 count=0 match=0 LEDG0=1 LEDR0=1 buzzer=0 saved_pass=1234 buffer=0000 lcd="NHẬP MÃ MỚI..."
[16000000 ns] Sau khi nhập mã mới 5678 : state=0 wrong_cnt=0 timer=12 count=0 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=0000 lcd="MỜI NHẬP MÃ"
PASS TEST 2: De lua mat khau moi 5678.
--- TEST 3: Nhập lại mã cũ 1234, yêu cầu phải sai ---
[22000000 ns] Sau khi nhập lại mã cũ 1234 : state=4 wrong_cnt=1 timer=11 count=4 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=1234 lcd="SAI MÃ! THỬ LẠI"
PASS TEST 3: Mã cũ bị từ chối, vào S_WRONG_CODE.
[23000000 ns] Sau khi bỏ qua WRONG_DELAY : state=0 wrong_cnt=1 timer=100000002 count=0 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=0000 lcd="MỜI NHẬP MÃ"
--- TEST 4: Nhập mã mới 5678 ---
[28000000 ns] Sau khi nhập mã mới 5678 : state=3 wrong_cnt=0 timer=11 count=4 match=1 LEDG0=1 LEDR0=0 buzzer=0 saved_pass=5678 buffer=5678 lcd="CỬA ĐÃ MỞ!"
PASS TEST 4: Mật khẩu mới đúng -> cửa mở.
--- TEST 5: Sai 3 lần để kiểm tra báo động ---
[29600000 ns] Sau khi bỏ qua UNLOCK_DELAY, san sang test hao dung : state=0 wrong_cnt=0 timer=500000002 count=0 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=0000 lcd="MỜI NHẬP MÃ"
[35600000 ns] Sai lần 1 : state=4 wrong_cnt=1 timer=11 count=4 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=0000 lcd="SAI MÃ! THỬ LẠI"
[36400000 ns] Sau WRONG_DELAY lần 1 : state=0 wrong_cnt=1 timer=100000002 count=0 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=0000 lcd="MỜI NHẬP MÃ"
[42400000 ns] Sai lần 2 : state=4 wrong_cnt=2 timer=11 count=4 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=1111 lcd="SAI MÃ! THỬ LẠI"
[43200000 ns] Sau WRONG_DELAY lần 2 : state=0 wrong_cnt=2 timer=100000002 count=0 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=0000 lcd="MỜI NHẬP MÃ"
[49200000 ns] Sai lần 3 : state=4 wrong_cnt=3 timer=11 count=4 match=0 LEDG0=0 LEDR0=1 buzzer=0 saved_pass=5678 buffer=2222 lcd="SAI MÃ! THỬ LẠI"
[50000000 ns] Sau WRONG_DELAY lần 3 : state=5 wrong_cnt=3 timer=100000002 count=4 match=0 LEDG0=0 LEDR0=1 buzzer=1 saved_pass=5678 buffer=2222 lcd="HỆ THỐNG KHOA!"
PASS TEST 5: Sai 3 lần -> vào S_ALARM.
===== KẾT THÚC MÔ PHỎNG =====
# Break in Module tb_smart_lock at D:/CE213/danh/Project (1) (1)/Project (1)/Project/th_smart_lock.v line 198
# Simulation Breakpoint: Break in Module tb_smart_lock at D:/CE213/danh/Project (1) (1)/Project (1)/Project/th_smart_lock.v line 198
# HMCRO ./Project_run_main_rci_verilog.do FAILED at line 24
```

Ảnh.17 Kết quả mô phỏng hiển thị console

Từ transcript có thể thấy hệ thống khởi tạo đúng sau reset với mật khẩu mặc định 1234. Khi nhập đúng 1234, hệ thống chuyển sang trạng thái mở cửa. Sau khi đổi mật khẩu sang 5678, mã cũ 1234 bị từ chối, còn mã mới 5678 mở cửa thành công. Ở lần kiểm tra bảo mật, khi nhập sai ba lần liên tiếp, wrong_cnt tăng đến 3, state chuyển sang 5 tương ứng S_ALARM, buzzer bằng 1 và LCD hiển thị HE THONG KHOA!.

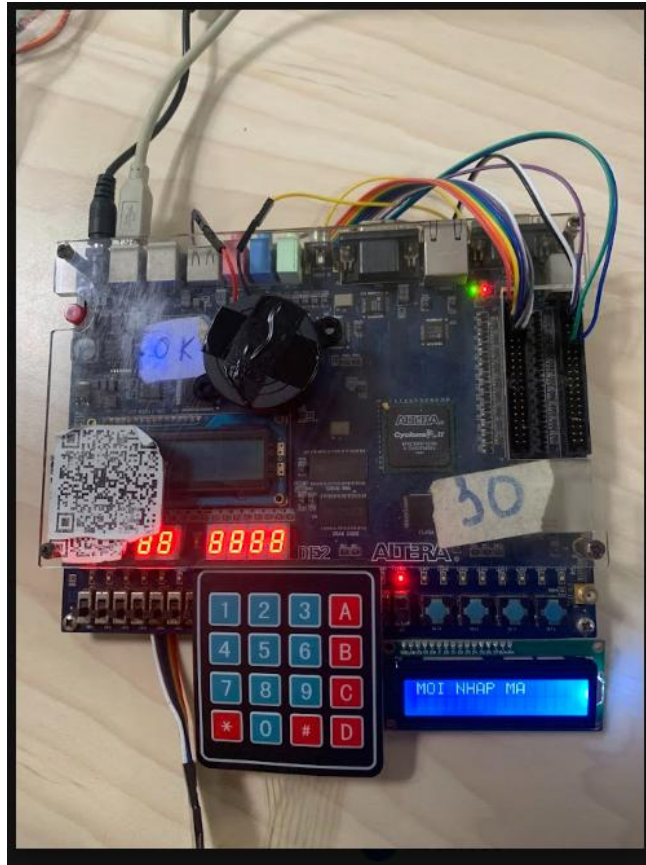


Ảnh.18 Kết quả mô phỏng dạng sóng

Trên waveform, tín hiệu CLOCK_50 chạy ổn định. Khi hệ thống ở trạng thái khóa, LED đỏ sáng và LED xanh tắt. Khi nhập đúng mật khẩu, LEDG0 chuyển lên mức 1 và LEDR0 tắt, thể hiện cửa được mở. Sau thời gian mở cửa, hệ thống quay lại trạng thái khóa. Các tín hiệu GPIO xuất hiện trạng thái Z do là chân giao tiếp ngoại vi/inout trong mô phỏng, không ảnh hưởng đến việc đánh giá chức năng chính của hệ thống.

2. Kết quả nạp

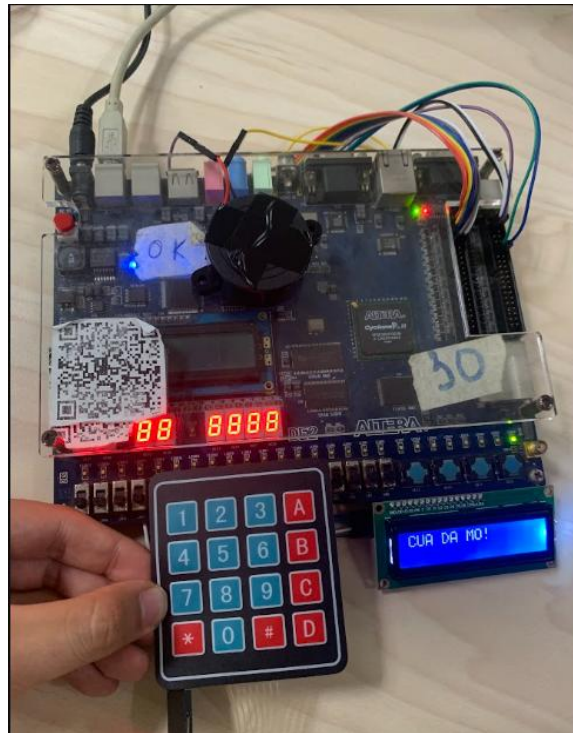
Bên dưới là loạt hình ảnh hệ thống được chụp lại với đầy đủ chức năng



Ảnh.19 Toàn bộ hệ thống Khóa số điện tử



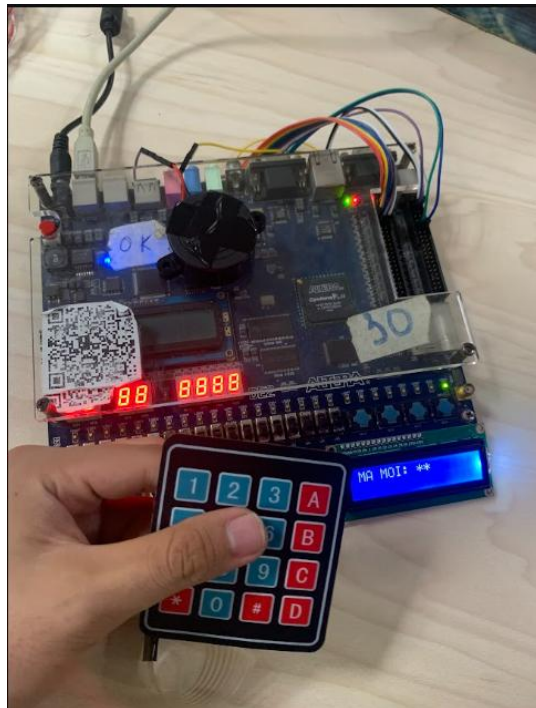
Ảnh.20 Trạng thái đang nhập mã



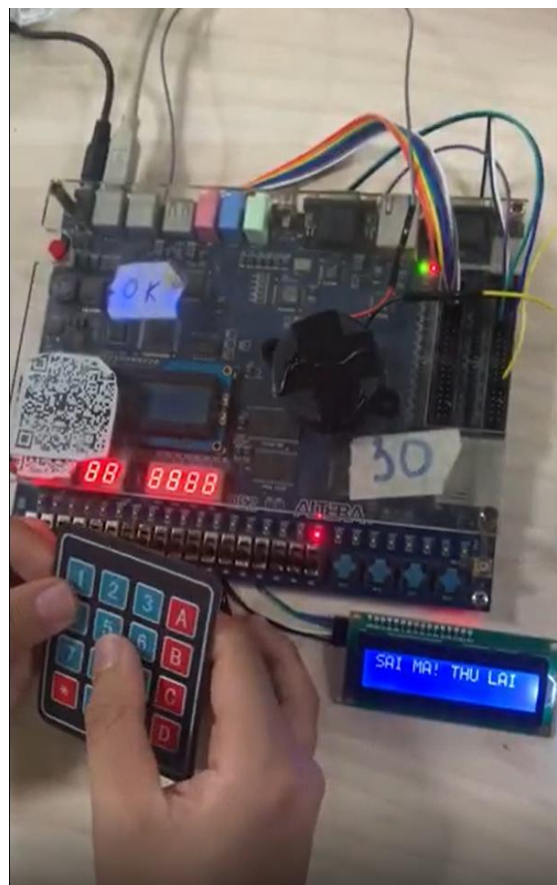
Ảnh.21 Trạng thái đã mở cửa



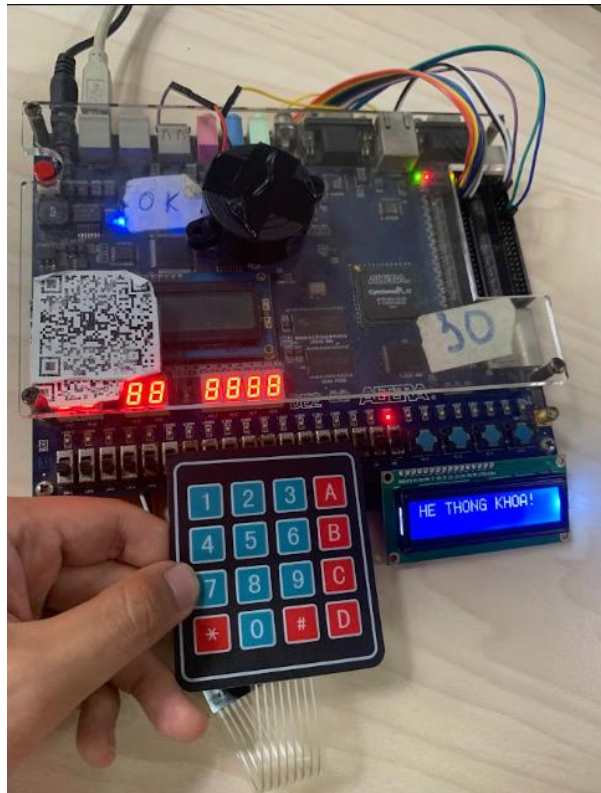
Ảnh.22 Trạng thái chờ nhập mã mới (đổi mật khẩu)



Ảnh.23 Trạng thái đang nhập mã mới



Ảnh.24 Trạng thái nhập sai mật khẩu (lần 1 và 2)



Ảnh.25 Trạng thái khóa vì nhập sai 3 lần

Link Video Demo

https://drive.google.com/file/d/1SsFjhPpZzo50XnHkNxx1zMmivss_M_Vd/view?usp=sharing

CHƯƠNG V: KẾT LUẬN

1. Kết luận đề tài

Đồ án đã hiện thực hóa thành công hệ thống Khóa số điện tử hoạt động ổn định theo thời gian thực dựa trên nền tảng chip FPGA của Kit phát triển Altera DE2 bằng ngôn ngữ Verilog HDL. Nhóm thực hiện đã hoàn thành toàn diện các mục tiêu thiết kế từ việc mô phỏng chính xác mạch logic trên phần mềm đến việc kết nối phần cứng mượt mà với các linh kiện ngoại vi thông qua cổng giao tiếp GPIO. Hệ thống đạt được độ tin cậy cao khi xử lý nhạy bén dữ liệu nhập từ Keypad 4x4 nhờ khối chống rung phím tốt, hiển thị các chuỗi ký tự và thông báo trạng thái trực quan lên màn hình LCD 16x2 qua giao thức nối tiếp I2C, thực hiện đóng/mở chốt cửa an toàn, cho phép linh hoạt thay đổi mã số mới và lập tức kích hoạt còi báo Buzzer vang dội để cảnh báo an ninh khi phát hiện hành vi nhập sai mật mã quá 3 lần quy định.

2. Hướng phát triển

Về hướng phát triển, hệ thống có thể được tối ưu hóa bằng cách tích hợp driver giao tiếp với bộ nhớ Flash hoặc EEPROM tích hợp sẵn trên Kit DE2 nhằm lưu trữ mật khẩu vĩnh viễn, giúp dữ liệu không bị mất hoặc reset về mặc định mỗi khi hệ thống ngắt nguồn điện completely. Bên cạnh đó, nhóm có thể nâng cấp giao diện hiển thị trực quan hơn thông qua cổng VGA ra màn hình lớn thay vì giới hạn trên LCD 16x2, đồng thời nghiên cứu kết hợp thêm các giải pháp bảo mật đa lớp hiện đại như module cảm biến vân tay, công nghệ thẻ từ RFID hoặc kết nối không dây IoT (Wi-Fi/Bluetooth) để giám sát và điều khiển đóng mở cửa từ xa qua điện thoại thông minh.